

Final Review

COMP4551

Binhang Yuan

Please Submit the Course Feedback



Thank you!

<https://sfq-survey.ust.hk/>

What We Have Explored.

Date	Topic
W1 - 02/03, 02/05	Introduction and Logistics & ML Preliminary
W2 - 02/10, 02/12	Stochastic Gradient Descent & Automatic Differentiation
W4 - 02/24, 02/26	Language Model Architecture & Large Scale Pretrain Overview
W5 - 03/03, 03/05	Nvidia GPU Performance & Collective Communication Library
W6 - 03/10, 03/12	Data-, Pipeline- Parallel Training & Tensor Model-, Optimizer- Parallel Training
W7 - 03/17	Sequence-, MoE- parallelism

About 30% in Final

Date	Topic
W8 - 03/26	Generative Inference
W9 - 03/31, 04/02	Inference Algorithm Optimizations & Inference System Optimizations
W10 - 04/07, 04/09	Spring Break & Prompt Engineering
W11 - 04/14, 04/16	Inference Scaling & Retrieval Augmented Generation
W12 - 04/21, 04/23	LLM Agent & Parameter Efficient Fine-Tuning
W13 - 04/28, 04/30	RL Alignment & LLM Evaluation

About 70% in Final

ML System Basics

ML System Basics

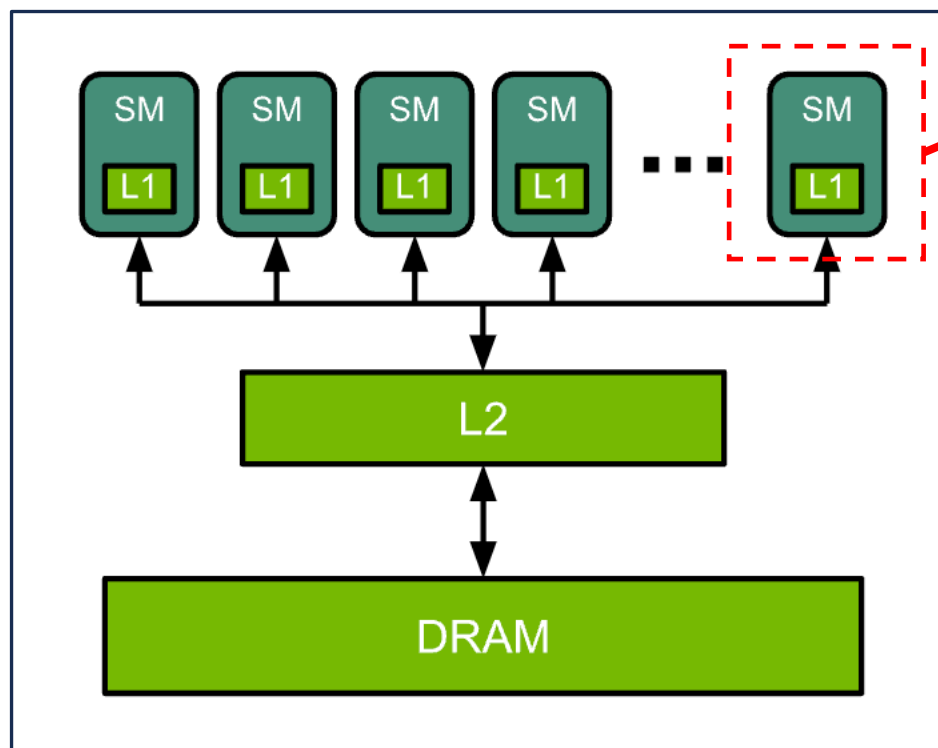
- Machine learning preliminary & PyTorch tensors:
 - Einstein notation in PyTorch.
- Stochastic gradient descent:
 - Define the empirical risk;
 - Optimizing the empirical loss.
- Automatic differentiation:
 - Forward mode AD;
 - Reverse Mode AD;
 - Compute the gradient of a Linear Layer.

Summary of a Linear Layer Computation

- Forward computation of a linear layer: $\mathbf{Y} = \mathbf{XW}$
 - Given input: $\mathbf{X} \in \mathbb{R}^{B \times D_1}$
 - Given weight matrix: $\mathbf{W} \in \mathbb{R}^{D_1 \times D_2}$
 - Compute output: $\mathbf{Y} \in \mathbb{R}^{B \times D_2}$
- Backward computation of a linear layer:
 - Given gradients w.r.t output: $\frac{\partial L}{\partial \mathbf{Y}} \in \mathbb{R}^{B \times H_2}$
 - Compute gradients w.r.t weight matrix: $\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^T \frac{\partial L}{\partial \mathbf{Y}} \in \mathbb{R}^{B \times H_2}$
 - Compute gradients w.r.t input: $\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \mathbf{W}^T \in \mathbb{R}^{B \times H_2}$

GPU Computation and Communication

Ampere GPU Architecture

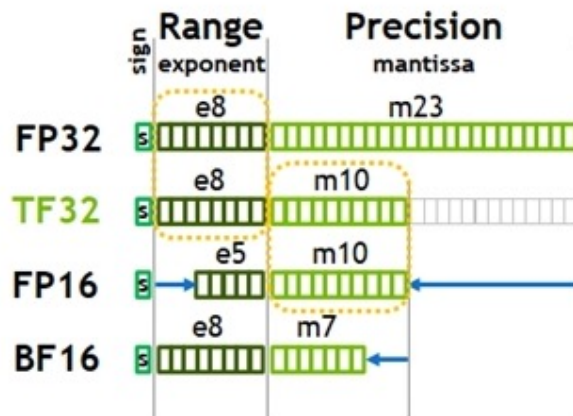


108 SM in a A100 GPU



A100 GPU Tensor Core Computation

- Multiply-add is the most frequent operation in modern neural networks. This is known as the fused multiply-add (FMA) operation.
- Includes one multiply operation and one add operation, counted as two float operations.
- A100 GPU has 1.41 GHz clock rate.
- The Ampere A100 GPU Tensor Cores multiply-add operations per clock:



Ampere A100 GPU FMA per clock on a SM					
FP64	TF32	FP16	INT8	INT4	INT1
64	512	1024	2048	4096	16384

Arithmetic Intensity

- Thus, on a given processor a given algorithm is math limited if:
 - $T_{math} > T_{mem}$
 - $\frac{\#op}{BW_{math}} > \frac{\#bytes}{BW_{mem}}$
- By simple algebra, the inequality can be rearranged to:
 - $\frac{\#op}{\#bytes} > \frac{BW_{math}}{BW_{mem}}$
- The left-hand side: the algorithm's arithmetic intensity.
- The right-hand side: ops:byte ratio.

NCCL Operators

- Optimized collective communication library from Nvidia to enable high-performance communication between CUDA devices.
- NCCL Implements:
 - **AllReduce;**
 - **Broadcast;**
 - **Reduce;**
 - **AllGather;**
 - **Scatter;**
 - **Gather;**
 - **ReduceScatter;**
 - **AlltoAll.**
- Easy to integrate into DL framework (e.g., PyTorch).

Ring based AllReduce

- In ring based AllReduce, we assume:
 - N : bytes to aggregate
 - B : bandwidth of each link
 - k : number of GPUs
 - The original tensor is equally split into k chunks.
- Ring based AllReduce implementation has two phases:
 - Reduction phrase (Aggregation phrase);
 - AllGather Phrase.
 - Total time: $\frac{2(k-1)N}{kB}$.

Language Model

Autoregressive Language Models

- A common way to write the joint distribution $p(x_{1:L})$ of a sequence to $x_{1:L}$ is using the chain rule of probability:

$$p(x_{1:L}) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \dots p(x_L|x_{1:L-1}) = \prod_{i=1}^L p(x_i|x_{1:i-1})$$

- In particular, $p(x_i|x_{1:i-1})$ is a conditional probability distribution of the next token x_i given the previous tokens $x_{1:i-1}$.
- An autoregressive language model is one where each conditional distribution $p(x_i|x_{1:i-1})$ can be computed efficiently (e.g., using a feedforward neural network).

TransformerBlocks($x_{1:L} \in \mathbb{R}^{L \times D}$) $\rightarrow \mathbb{R}^{L \times D}$

- B is the batch size;
- L is the sequence length;
- D is the model dimension;
- Multi-head attention:
 $D = n_h \times H$
- H is the head dimension;
- n_h is the number of heads.

Computation	Input	Output
$Q = xW^Q$	$x \in \mathbb{R}^{L \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q \in \mathbb{R}^{L \times D}$
$K = xW^K$	$x \in \mathbb{R}^{L \times D}, W^K \in \mathbb{R}^{D \times D}$	$K \in \mathbb{R}^{L \times D}$
$V = xW^V$	$x \in \mathbb{R}^{L \times D}, W^V \in \mathbb{R}^{D \times D}$	$V \in \mathbb{R}^{L \times D}$
$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{L \times D}$	$Q_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$
$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{L \times D}$	$K_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$
$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{L \times D}$	$V_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots, n_h$	$Q_i, K_i \in \mathbb{R}^{L \times H}$	$\text{score}_i \in \mathbb{R}^{L \times L}$
$Z_i = \text{score}_i V_i, i = 1, \dots, n_h$	$\text{score}_i \in \mathbb{R}^{L \times L}, V_i \in \mathbb{R}^{L \times H}$	$Z_i \in \mathbb{R}^{L \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{L \times H}, i = 1, \dots, n_h$	$Z \in \mathbb{R}^{L \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{L \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{L \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{L \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{L \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{L \times 4D}$	$A' \in \mathbb{R}^{L \times 4D}$
$x' = A'W^2$	$A' \in \mathbb{R}^{L \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$x' \in \mathbb{R}^{L \times D}$

Scaling Law

- Intuitively:
 - Increase parameter # $N \rightarrow$ better performance
 - Increase dataset scale $D \rightarrow$ better performance
- But we have a fixed computational budget on $C \approx 6ND$
- *To maximize model performance, how should we allocate C to N and D ?*



Training Compute-Optimal Large Language Models

Jordan Hoffmann*, Sebastian Borgeaud*, Arthur Mensch*, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals and Laurent Sifre*

*Equal contributions

We investigate the optimal model size and number of tokens for training a transformer language model under a given compute budget. We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. By training over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens, we find that for compute-optimal training, the model size and the number of training tokens should be scaled equally: for every doubling of model size the number of training tokens should also be doubled. We test this hypothesis by training a predicted compute-optimal model, *Chinchilla*, that uses the same compute budget as *Gopher* but with 70B parameters and 4× more data. *Chinchilla* uniformly and significantly outperforms *Gopher* (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B) on a large range of downstream evaluation tasks. This also means that *Chinchilla* uses substantially less compute for fine-tuning and inference, greatly facilitating downstream usage. As a highlight, *Chinchilla* reaches a state-of-the-art average accuracy of 67.5% on the MMLU benchmark, greater than a 7% improvement over *Gopher*.

Evaluating Distributed Computation

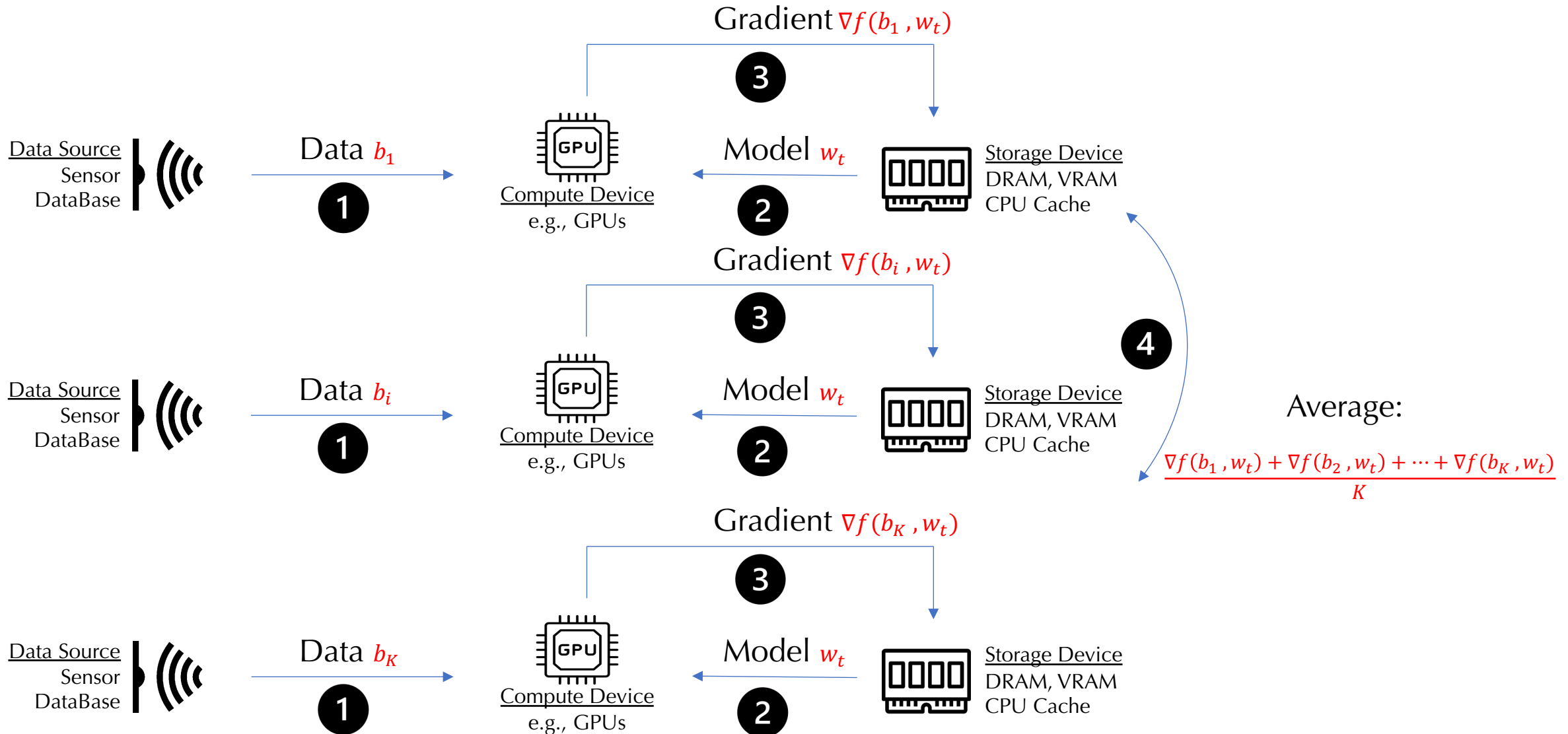
- Scaling law tells us given a fixed computation budget, how should we decide the model scale and data corpus.
- The computation budget is formulated by the total FLOPs demanded during the computation.
- But the GPU cannot usually work at its peak FLOPs.
- How can we evaluate the performance of a distributed training workflow?
 - Training throughput (token per second);
 - Scalability;
 - Model FLOPs Utilization.

Parallel Training

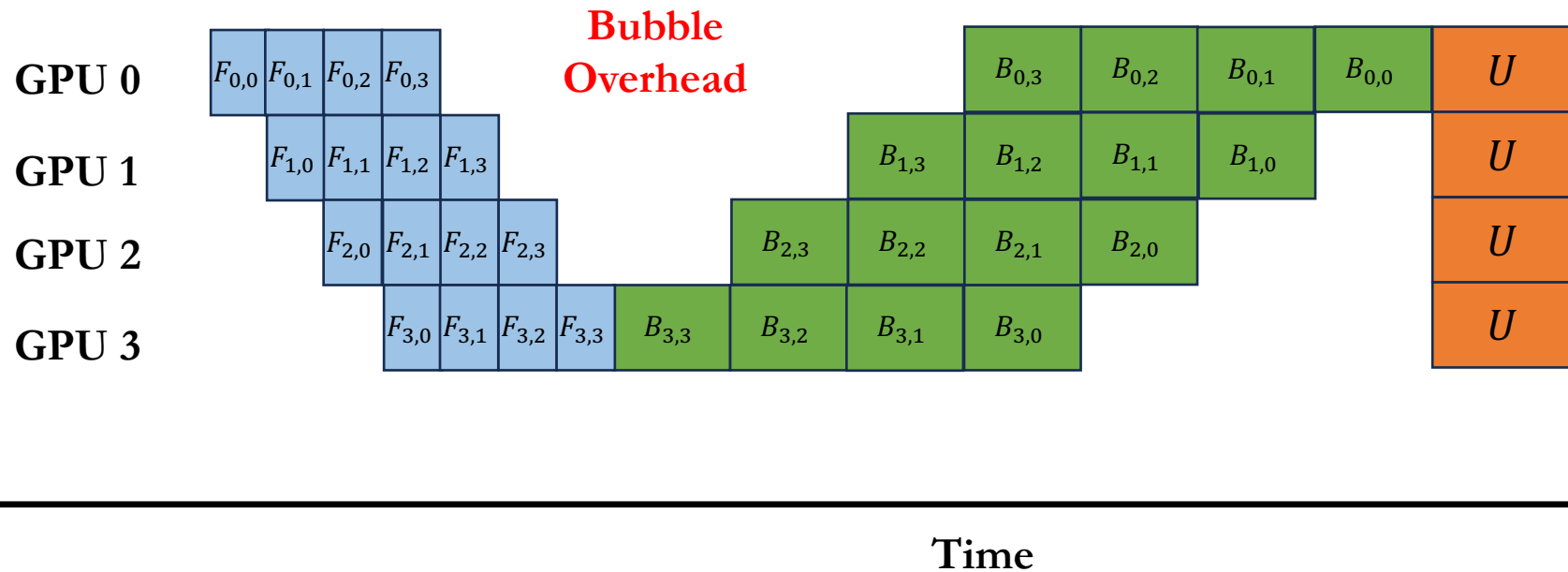
Parallel Training

- Categories:
 - Data parallelism;
 - Pipeline parallelism;
 - Tensor model parallelism;
 - Optimizer parallelism.
 - MoE parallelism;
 - Parallelism for long sequence.
- What are the communication paradigms?
 - Communication targets;
 - (Collective) Communication operator(s);
 - Communication volume (i.e., corresponding number of bytes of the communication targets).
- What are their advantages and disadvantages?
- Still important in the final.

Data Parallel SGD



Pipeline Parallelism - GPipe

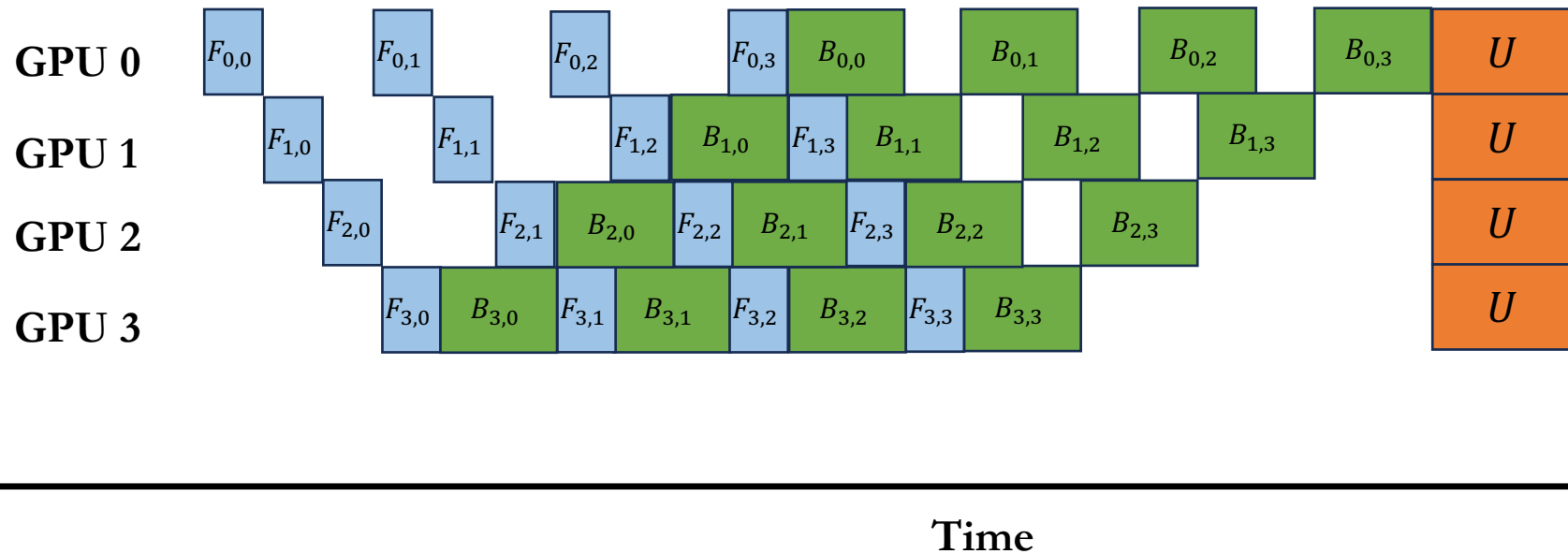


- If we ignore the computation time of optimizer updates.
- Suppose:
 - K is the number of GPUs;
 - M is the number of micro-batches;
- What is the percentage of bubble overhead? $\frac{K-1}{M+K-1}$

The number on each block represents the stage index and the micro-batch index.

Pipeline Parallelism - 1F1B

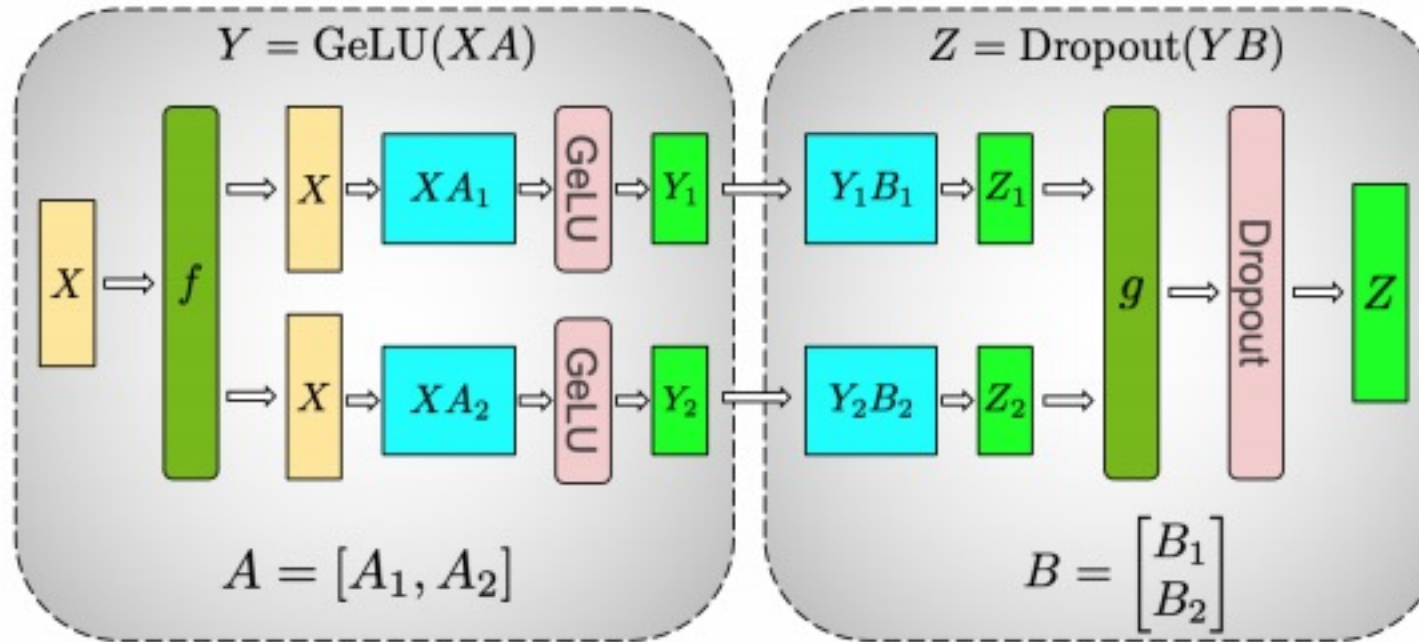
**Bubble
Overhead**



- If we ignore the computation time of optimizer updates.
- Suppose:
 - K is the number of GPUs;
 - M is the number of micro-batches;
- What is the percentage of bubble overhead? $\frac{K-1}{M+K-1}$

The number on each block represents the stage index and the micro-batch index.

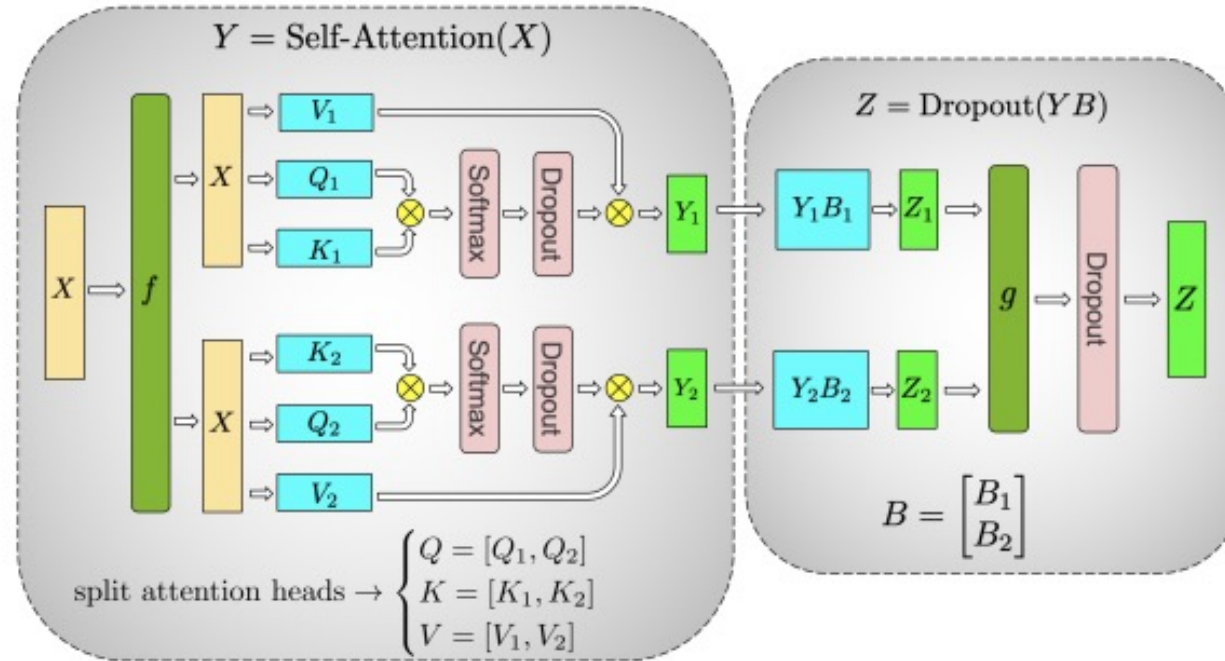
MLP in Tensor Model Parallelism



(a) MLP

- f is the identity operator in the forward pass and the **AllReduce** operator in the backward pass.
- g is the **AllReduce** operator in the forward pass and the identity operator in the backward pass.

Multi-Head Attention in Tensor Model Parallelism



(b) Self-Attention

- f is the identity operator in the forward pass and the **AllReduce** operator in the backward pass.
- g is the **AllReduce** operator in the forward pass and the identity operator in the backward pass.

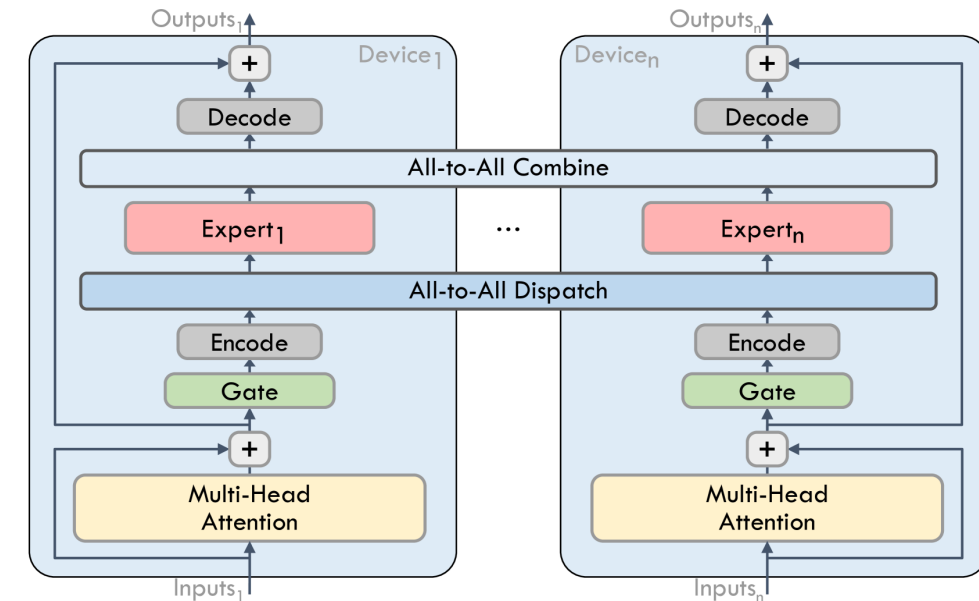
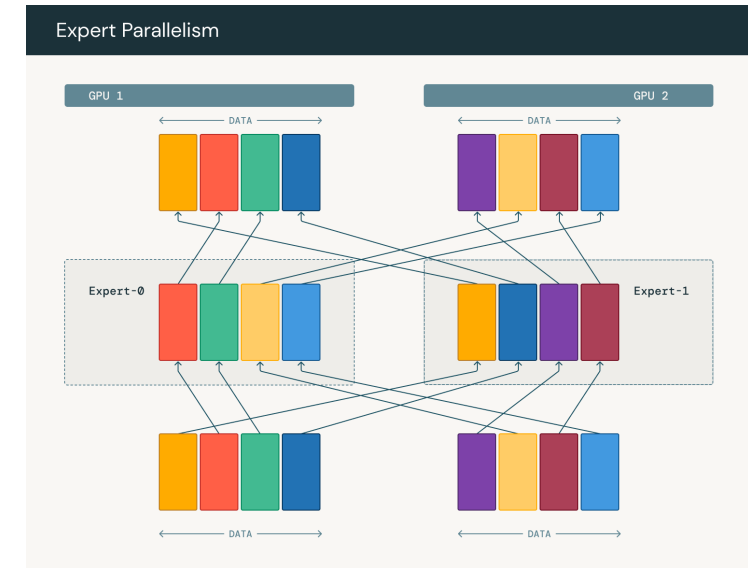
Zero Redundancy Optimizer (ZeRO)

	gpu ₀	...	gpu _i	...	gpu _{N-1}	Memory Consumed
Baseline		...		...		$(2 + 2 + K) * \Psi$
P _{os}		...		...		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$
P _{os+g}		...		...		$2\Psi + \frac{(2 + K) * \Psi}{N_d}$
P _{os+g+p}		...		...		$\frac{(2 + 2 + K) * \Psi}{N_d}$

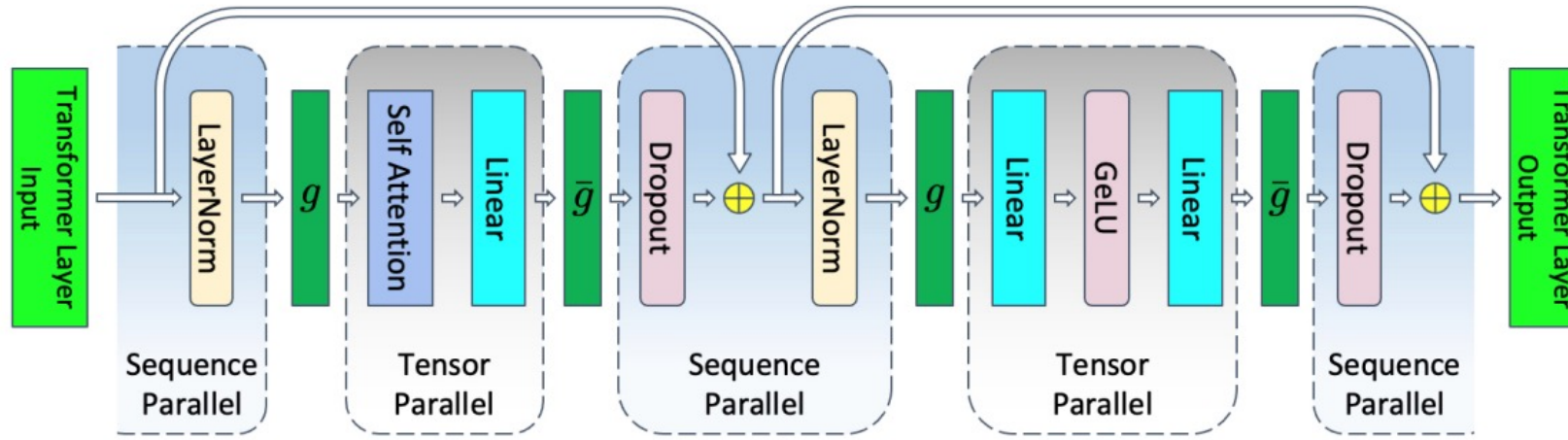
- Ψ is the total number of parameters;
- K denotes the memory multiplier of optimizer states;
- N_d denotes the parallel degree.

MoE Parallelism

- In the forward pass:
 - Two **AlltoAll** operations to communicate the activations:
 - One **AlltoAll** operation dispatches the routed activations in the current data batch to the corresponding experts;
 - One **AlltoAll** operation combines the computed expert output in the data batch that needs to be processed by the device.
 - Notice that there could be a little additional overhead to first share the shape of the dispatch tensors.
- In the backward pass:
 - Two **AlltoAll** operations to communicate the corresponding gradients of the activations back to the original device.
 - **AllReduce** operations to synchronize the gradients of non-expert model weights.



Megatron Sequence Parallelism



In tensor model parallelism combined with sequence parallelis:

g is **AllGather** operation in the forward pass and **ReduceScatter** in the backward pass.

$\bar{g}$ is **ReduceScatter** in the forward pass and **AllGather** operation in the backward pass.

- Sequence parallelism is an extension of the previous tensor model parallelism:
- Partitioning along the sequence dimension reduces the memory required for the activations. This extra level of parallelism introduces new collective communication paradigm.

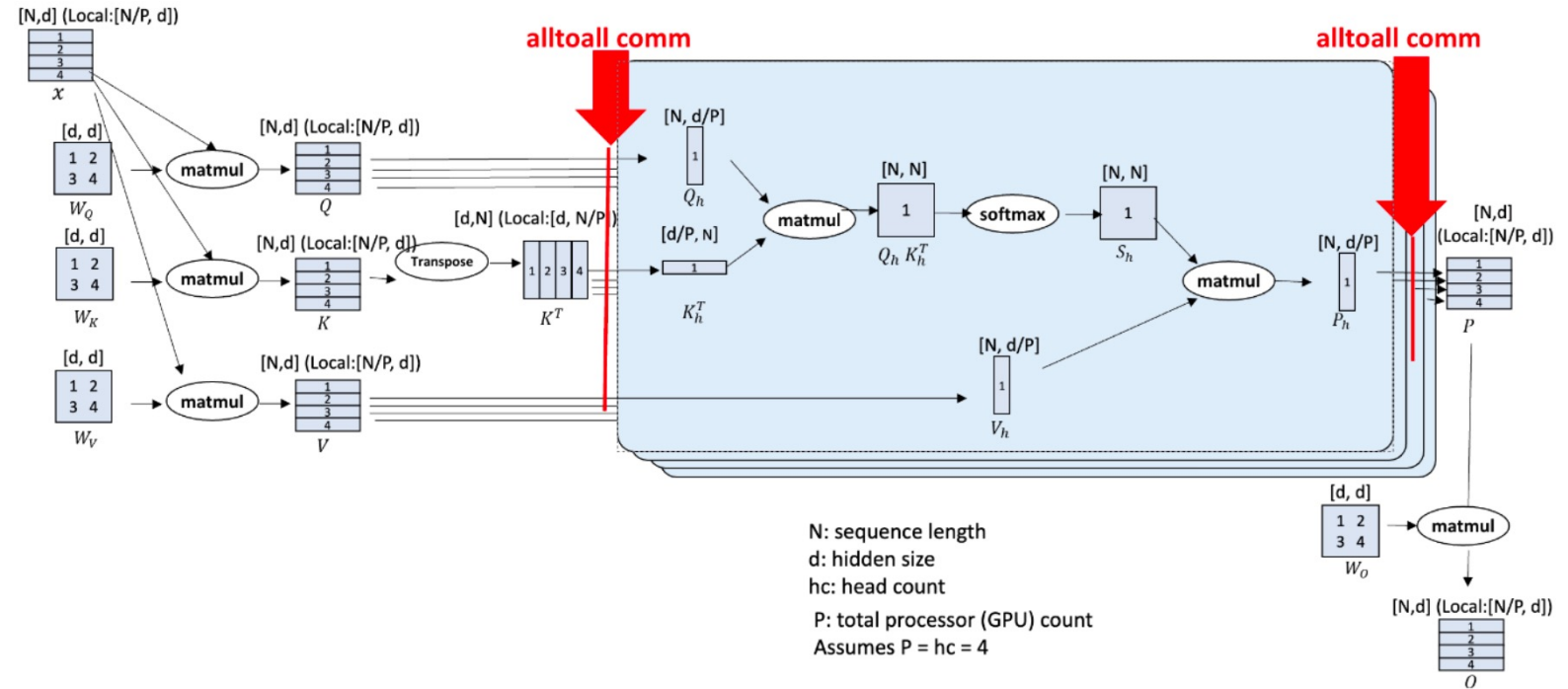
Deepspeed-Ulysses



RELAXED
SYSTEM LAB

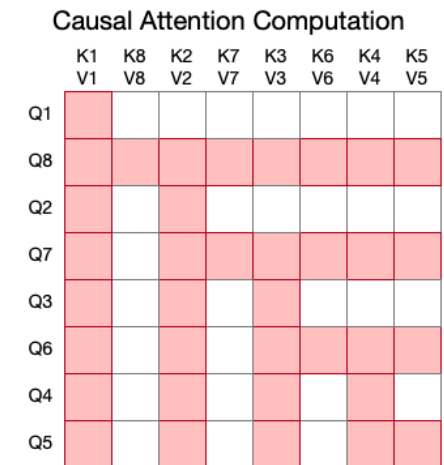
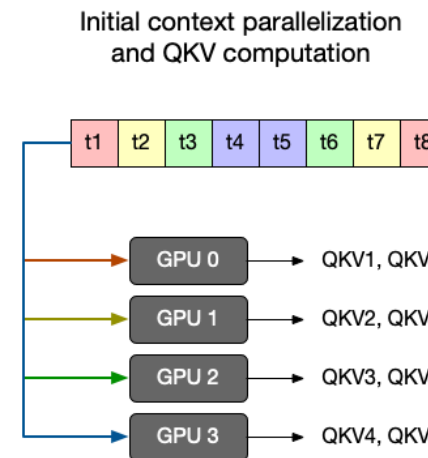
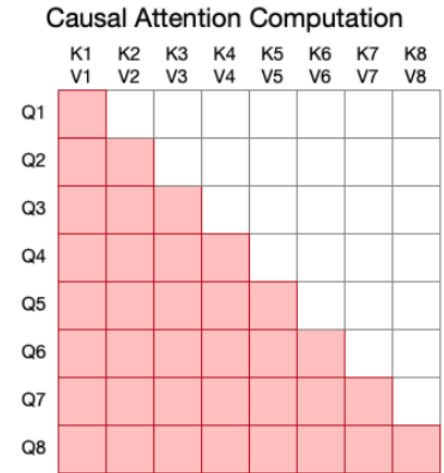
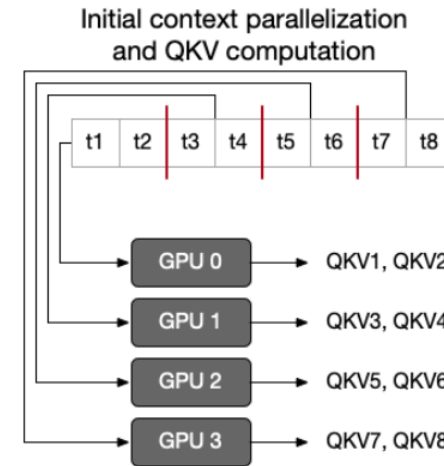
• Core idea:

- Partition the computation of attention heads among different GPUs;
- Use one All-to-All to shuffle the input Query, Key, Value;
- Use one All-to-All to collect the corresponding output.



Ring Attention/Megatron Context Parallelism

- Ring attention computation for the attention mechanism:
 - Split the data by sequence dimension;
 - Iteratively accumulate local attention results by iterating Q and K/V as a nested loop;
 - Each GPU holds a portion of queries, and sends its keys and values to the next GPU and receives them from the previous GPU (ring).
- A trivial issue to be fixed in Ring Attention: the load-balance in each iteration:
 - Megatron context parallelism implementation: an optimization not to compute unnecessary masked parts to balance the workflow.



<https://insujang.github.io/2024-09-20/introducing-context-parallelism>

Generative Inference

Autoregressive Generation

- Autoregressive generation with two phrases computation:
 - **Prefill phrase:**
 - The model takes a prompt sequence as input and engages in the generation of a key-value cache (KV cache) for each Transformer layer.
 - Computation bounded.
 - **Decode phrase:**
 - For each decode step, the model updates the KV cache and reuses the KV to compute the output.
 - IO bounded.
- Parallel Generative Inference:
 - Pipeline parallelism;
 - Tensor model parallelism.



Prefill: TransformerBlocks($X \in \mathbb{R}^{L \times D}$) $\rightarrow X' \in \mathbb{R}^{L \times D}$

For each inference request:

- ~~$B = 1$~~ ;
- L is the input sequence length;
- D is the model dimension;
- Multi-head attention:
 $D = n_H \times H$
- H is the head dimension;
- n_h is the number of heads.

Computation	Input	Output
$Q = XW^Q$	$X \in \mathbb{R}^{L \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q \in \mathbb{R}^{L \times D}$
$K = XW^K$	$X \in \mathbb{R}^{L \times D}, W^K \in \mathbb{R}^{D \times D}$	$K \in \mathbb{R}^{L \times D}$
$V = XW^V$	$X \in \mathbb{R}^{L \times D}, W^V \in \mathbb{R}^{D \times D}$	$V \in \mathbb{R}^{L \times D}$
$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{L \times D}$	$Q_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{L \times D}$	$K_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{L \times D}$	$V_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots n_h$	$Q_i, K_i \in \mathbb{R}^{L \times H}$	$\text{score}_i \in \mathbb{R}^{L \times L}$
$Z_i = \text{score}_i V_i, i = 1, \dots n_h$	$\text{score}_i \in \mathbb{R}^{L \times L}, V_i \in \mathbb{R}^{L \times H}$	$Z_i \in \mathbb{R}^{L \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{L \times H}, i = 1, \dots n_h$	$Z \in \mathbb{R}^{L \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{L \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{L \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{L \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{L \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{L \times 4D}$	$A' \in \mathbb{R}^{L \times 4D}$
$X' = A'W^2$	$A' \in \mathbb{R}^{L \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$X' \in \mathbb{R}^{L \times D}$

Generate the first token.

$$p(x_{L+1}|x_{1:L}) = \text{softmax}(X_L^{-1} W_{lm})$$

-1 indicates the last TransformerBlock

Decode: TransformerBlocks($t \in \mathbb{R}^{1 \times D}$) $\rightarrow t' \in \mathbb{R}^{1 \times D}$

For each inference request:

- $B = 1$;
- L is the current cached sequence length; it increases by 1 after each step.
- D is the model dimension;
- Multi-head attention:
 $D = n_H \times H$
- H is the head dimension;
- n_h is the number of heads.

Update the KV cache:

$$K = \text{concat}(K_{\text{cache}}, K_d)$$

$$V = \text{concat}(V_{\text{cache}}, V_d)$$

Generate the second token:

$$p(x_{L+2}|x_{1:L+1}) = \text{softmax}(T_{L+1}^{-1} W_{lm})$$

-1 indicates the last TransformerBlock

Computation	Input	Output
$Q = Q_d = TW^Q$	$T \in \mathbb{R}^{1 \times D}, W^Q \in \mathbb{R}^{D \times D}$	$Q, Q_d \in \mathbb{R}^{1 \times D}$
$K_d = TW^K$	$T \in \mathbb{R}^{1 \times D}, W^K \in \mathbb{R}^{D \times D}$	$K_d \in \mathbb{R}^{1 \times D}$
$K = \text{concat}(K_{\text{cache}}, K_d)$	$K_{\text{cache}} \in \mathbb{R}^{L \times D}, K_d \in \mathbb{R}^{1 \times D}$	$K \in \mathbb{R}^{(L+1) \times D}$
$V_d = tW^V$	$t \in \mathbb{R}^{1 \times D}, W^V \in \mathbb{R}^{D \times D}$	$V_d \in \mathbb{R}^{1 \times D}$
$V = \text{concat}(V_{\text{cache}}, V_d)$	$V_{\text{cache}} \in \mathbb{R}^{L \times D}, V_d \in \mathbb{R}^{1 \times D}$	$V \in \mathbb{R}^{(L+1) \times D}$
$[Q_1, Q_2 \dots, Q_{n_h}] = \text{Partition}_{-1}(Q)$	$Q \in \mathbb{R}^{1 \times D}$	$Q_i \in \mathbb{R}^{1 \times H}, i = 1, \dots, n_h$
$[K_1, K_2 \dots, K_{n_h}] = \text{Partition}_{-1}(K)$	$K \in \mathbb{R}^{(L+1) \times D}$	$K_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$[V_1, V_2 \dots, V_{n_h}] = \text{Partition}_{-1}(V)$	$V \in \mathbb{R}^{(L+1) \times D}$	$V_i \in \mathbb{R}^{(L+1) \times H}, i = 1, \dots, n_h$
$\text{Score}_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{H}}), i = 1, \dots, n_h$	$Q_i \in \mathbb{R}^{1 \times H}, K_i \in \mathbb{R}^{(L+1) \times H}$	$\text{score}_i \in \mathbb{R}^{1 \times (L+1)}$
$Z_i = \text{score}_i V_i, i = 1, \dots, n_h$	$\text{score}_i \in \mathbb{R}^{1 \times (L+1)}, V_i \in \mathbb{R}^{(L+1) \times H}$	$Z_i \in \mathbb{R}^{1 \times H}$
$Z = \text{Merge}_{-1}([Z_1, Z_2 \dots, Z_{n_h}])$	$Z_i \in \mathbb{R}^{1 \times H}, i = 1, \dots, n_h$	$Z \in \mathbb{R}^{1 \times D}$
$\text{Out} = ZW^O$	$Z \in \mathbb{R}^{1 \times D}, W^O \in \mathbb{R}^{D \times D}$	$\text{Out} \in \mathbb{R}^{1 \times D}$
$A = \text{Out} W^1$	$\text{Out} \in \mathbb{R}^{1 \times D}, W^1 \in \mathbb{R}^{D \times 4D}$	$A \in \mathbb{R}^{1 \times 4D}$
$A' = \text{relu}(A)$	$A \in \mathbb{R}^{1 \times 4D}$	$A' \in \mathbb{R}^{1 \times 4D}$
$T' = A'W^2$	$A' \in \mathbb{R}^{1 \times 4D}, W^2 \in \mathbb{R}^{4D \times D}$	$T' \in \mathbb{R}^{1 \times D}$

Customize Text Generation

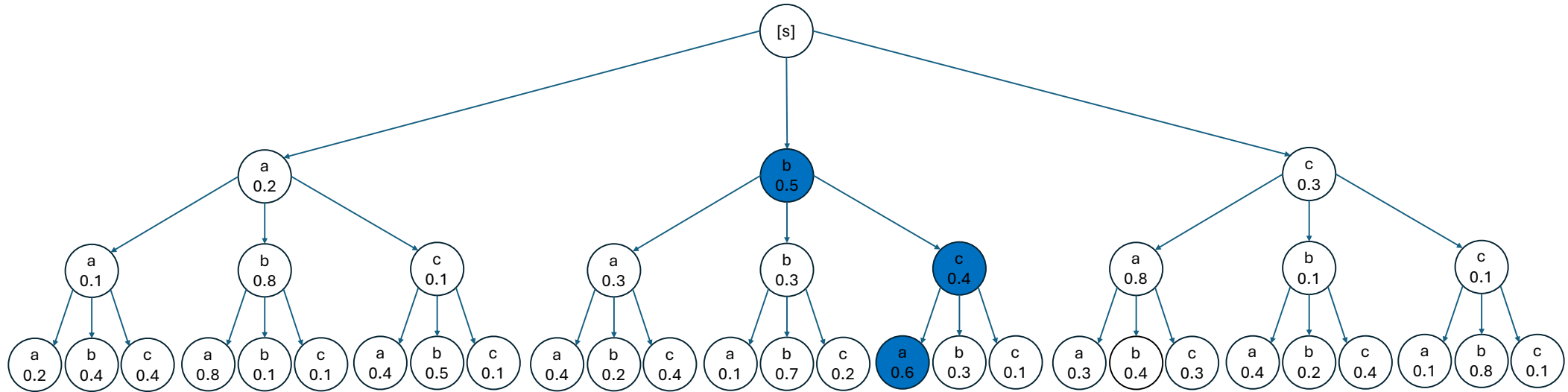
- **Greedy search**: in every generation step, keep the token with the highest probability.
- **Beam search**: always keeps k candidates and picks the best of the candidates at the end.
- **Sampling**: instead of deterministic selecting the largest tokens, we use a random number generator to sample tokens following the distribution computed by the LM.
 - **Top-k sampling**: only the k (e.g., $k = 6$) most likely next words are filtered to be sampled.
 - **Top-p sampling**: chooses from the smallest possible set of words whose cumulative probability exceeds the probability p (e.g., $p = 0.92$).

Sample Question

Suppose the calculated probability in a decode step is $\{t_1 : 0.6, t_2 : 0.2, t_3 : 0.1, t_4 : 0.05, t_5 : 0.03, t_6 : 0.02\}$, which of the following statements is correct? **C**

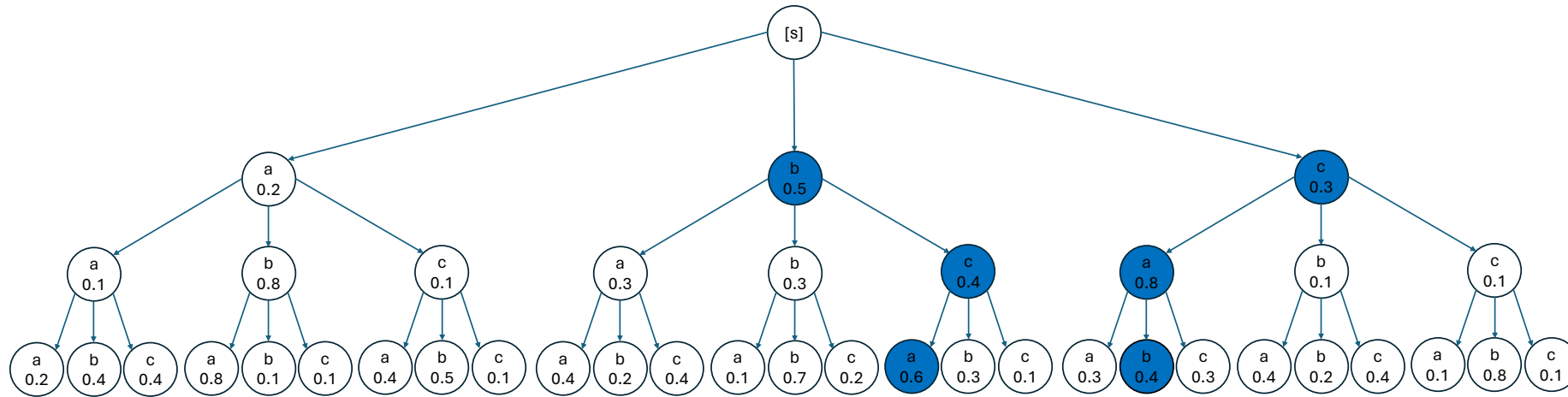
- A. When we apply top k sampling where $k = 2$, it is possible to generate t_3 in the current step.
- B. When we apply greedy search, it is possible to generate t_3 in the current step.
- C. When we apply top p sampling where $p = 0.85$, it is possible to generate t_3 in the current step.
- D. None of the above.

Homework 3-2 Greedy



- Solution:
 - b c a

Homework 3-2 Beam Search



- Solution:
 - b c a
 - c a b

Generative Inference Optimization

- Algorithm optimization:
 - “Slightly” change the original computation at its bottleneck to make it run much faster;
- Decrease the I/O volume:
 - Model compression, i.e., quantization;
 - Knowledge distillation.
 - KV cache optimization.

Generative Inference Optimization

- System optimization:
 - Improve the system efficiency without changing the original computation;
- General ideas
 - For each request, generate multiple tokens simultaneously:
 - Speculative decoding;
 - Parallel decoding.
 - For multiple requests, effectively batching them:
 - Continuous batching;
 - Disaggregated inference.

Sample Question

Which of the following statements is correct about the optimization of generative inference? **D**

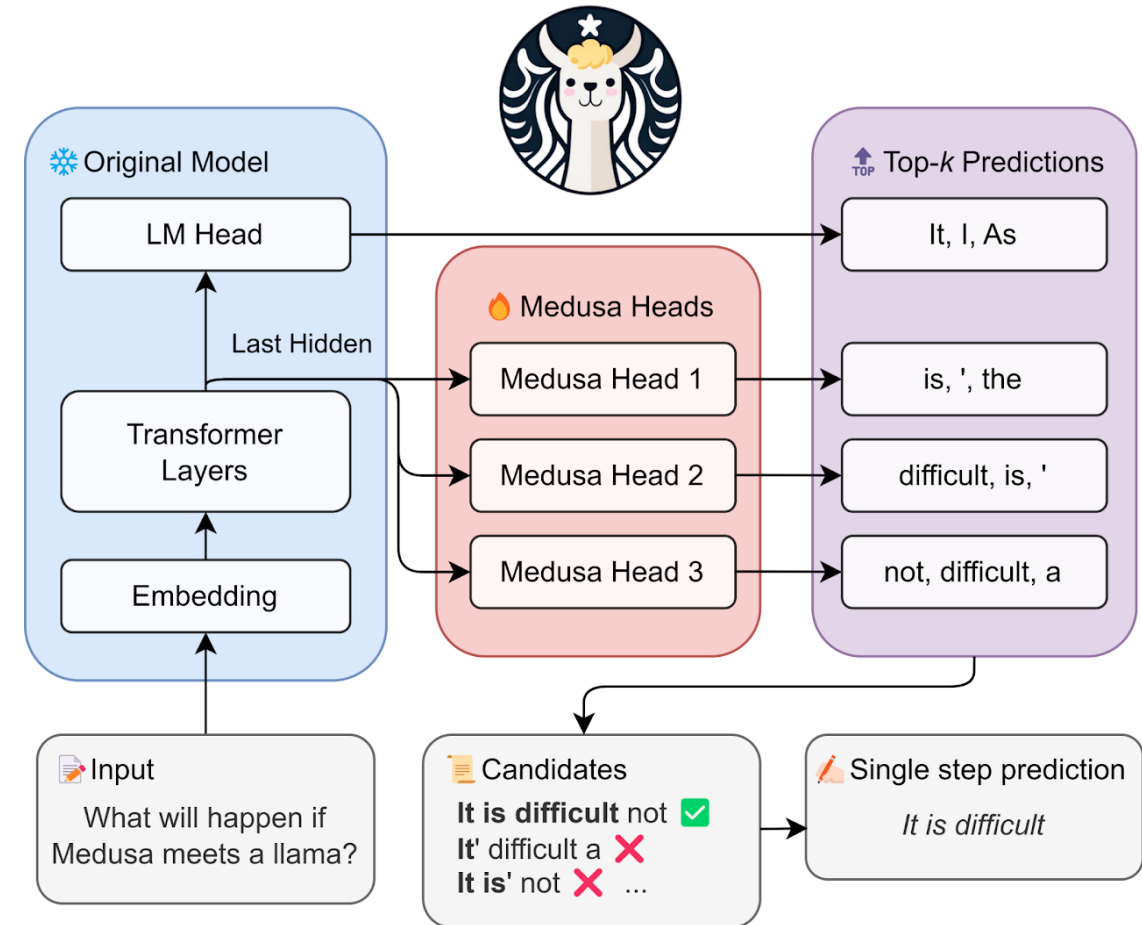
- A. Quantization-based methods mainly reduce the I/O cost.
- B. Speculative decoding method mainly increases the arithmetic intensity during the decode phase.
- C. Parallel decoding method mainly increases the arithmetic intensity during the decode phase.
- D. All of the above.

Speculative Decoding

- Observation:
 - A small *assistant/speculative model* very often generates the same tokens as the large original LLM (sometime s referred to as the *main/target model*).
- Speculative decoding overview:
 - The assistant model auto-regressively generates a sequence of N candidate tokens;
 - The candidate tokens are passed to the original LLM to be verified. The original model takes the candidate tokens as input and performs *a single forward pass*:
 - All candidate tokens up to the first mismatch are correct;
 - After the first mismatch, replace the first incorrect candidate token with the correct token from the main model (fox), and discard all predicted tokens that come after this mismatched token.
 - Repeat this process until the end condition is reached.

Parallel Decoding

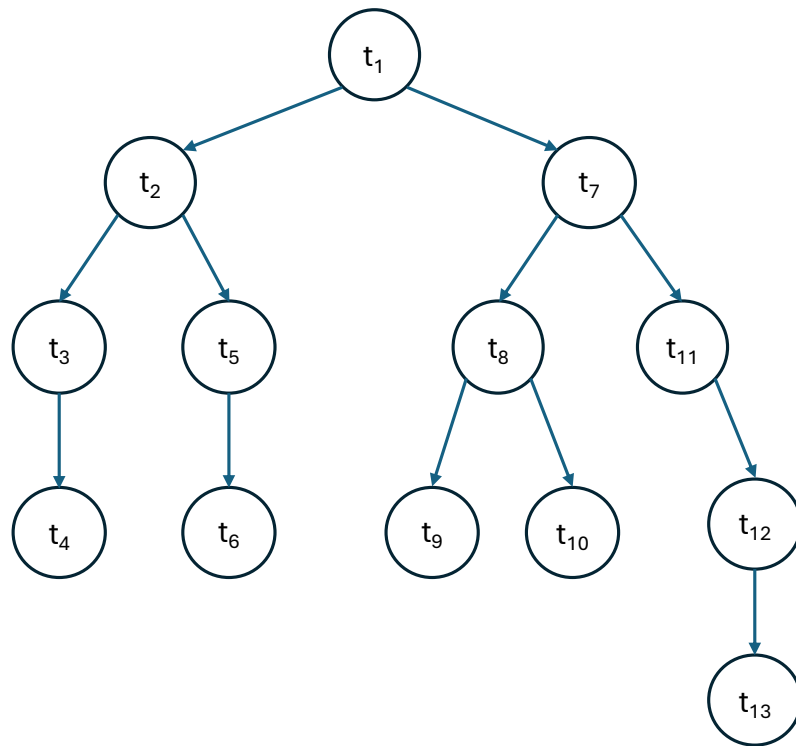
- Rather than pulling in an entirely new assistant model to predict subsequent tokens, Medusa simply extends the original LLM itself.
- Medusa heads are the additional decoding heads built on top of the last hidden states of the LLM, enabling the prediction of several subsequent tokens in parallel.
- Each Medusa head is a single layer of feed-forward network, augmented with a residual connection.
- Training these heads is straightforward: for a relatively small dataset, the original model remains static; only the Medusa heads are updated.



<https://www.together.ai/blog/medusa>

Homework 4-2 Tree Attention

- 2.1 Tree attention in speculative decoding solution:

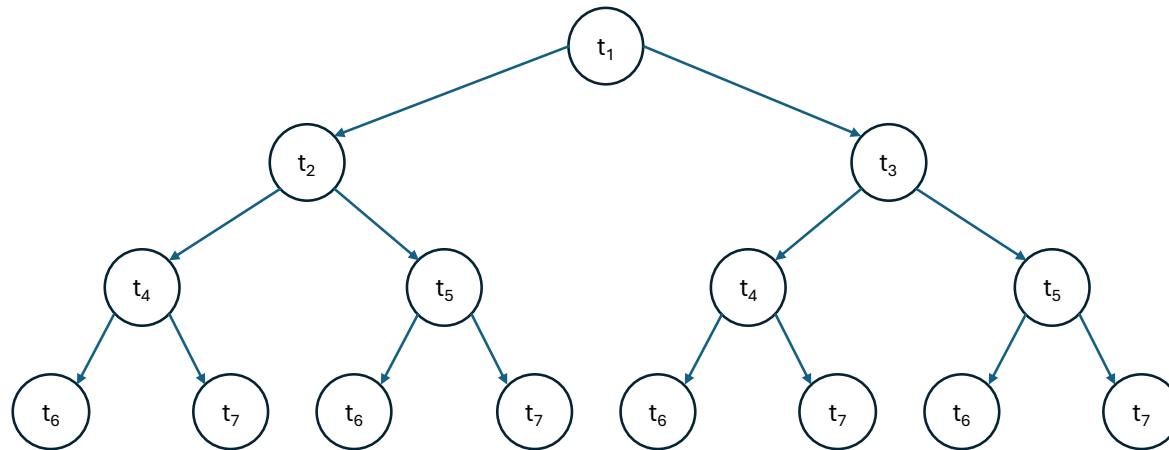


The solution is not unique; suppose we use a DFS traverse order, we have the answer:

	t1	t2	t3	t4	t5	t6	t7	t8	t9	t10	t11	t12	t13
t1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
t2		✓	✓	✓	✓	✓							
t3			✓	✓									
t4				✓									
t5					✓	✓							
t6						✓							
t7							✓	✓	✓	✓	✓	✓	✓
t8								✓	✓	✓			
t9									✓				
t10										✓			
t11											✓	✓	✓
t12												✓	✓
t13													✓

Homework 4-2 Tree Attention

- 2.2 Tree attention in parallel decoding solution:



The solution is not unique; suppose we use a level traverse order; we have the answer:

	t1	t2	t3	t4	t5	t4	t5	t6	t7	t6	t7	t6	t7	t6	t7
t1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
t2		✓		✓	✓			✓	✓	✓	✓				
t3			✓			✓	✓					✓	✓	✓	✓
t4				✓				✓	✓						
t5					✓					✓	✓				
t4						✓						✓	✓		
t5							✓							✓	✓
t6								✓							
t7									✓						
t6										✓					
t7											✓				
t6												✓			
t7													✓		
t6														✓	
t7															✓

Sample Question

Suppose we are using tree attention to improve the memory efficiency in speculative decoding, three candidate sequences are $S_1 : t_1, t_2, t_3, t_4$, $S_2 : t_1, t_2, t_5, t_6$, $S_3 : t_1, t_7, t_8$, which is the right mask matrix? **C**

- A. Mask matrix:

	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
t_1	✓	✓	✓	✓	✓	✓	✓	✓
t_2		✓	✓	✓				
t_3			✓	✓				
t_4				✓				
t_5					✓	✓		
t_6						✓		
t_7							✓	✓
t_8								✓

- B. Mask matrix:

	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
t_1	✓	✓	✓	✓	✓	✓		
t_2		✓	✓	✓	✓	✓		
t_3			✓	✓				
t_4				✓				
t_5					✓			
t_6						✓		
t_7							✓	✓
t_8								✓

- C. Mask matrix:

	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
t_1	✓	✓	✓	✓	✓	✓	✓	✓
t_2		✓	✓	✓	✓	✓		
t_3			✓	✓				
t_4				✓				
t_5					✓	✓		
t_6						✓		
t_7							✓	✓
t_8								✓

- D. None of the above mask matrices is correct.

Continuous Batching

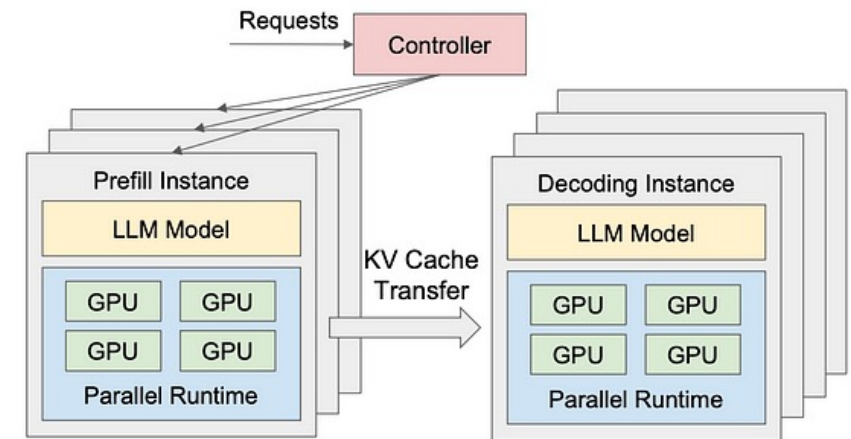
- Orca: <https://www.usenix.org/conference/osdi22/presentation/yu>
- Key idea:
 - Instead of waiting until every sequence in a batch has completed generation, Orca implements iteration-level scheduling;
 - Once a sequence in a batch has completed generation, a new sequence can be inserted in its place;
 - Higher GPU utilization than static batching.

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					
S_4	S_4	S_4					

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Disaggregated Inference

- DistServe:
<https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf>
- Key ideas:
 - Prefill computation on some GPUs;
 - Decoding computation on some other GPUs;
 - Prefill and decoding instances can have different parallel configurations;
 - Dynamic configuration of the prefill / decoding ratio;
 - Overhead: KV-cache communication.



Improve the Generative Inference Quality

Prompt Engineering

- ***Prompt engineering*** is the practice of developing and optimizing prompts to efficiently use large language models (LLMs) for a variety of applications.
- Rules of Thumb from OpenAI.
- Advanced prompt engineering:
 - Baseline: zero-shot prompting;
 - Few-shot prompting;
 - Chain-of-thought (CoT);
 - Self-consistency;
 - Tree of Thoughts (ToT);
 - Graph of Thoughts (GoT);
 - Agent skills.
- Model safety issues:
 - Prompt injection;
 - Prompt leaking;
 - Jailbreaking.

Sample Question

Which of the following suggestions is recommended in prompt engineering? **D**

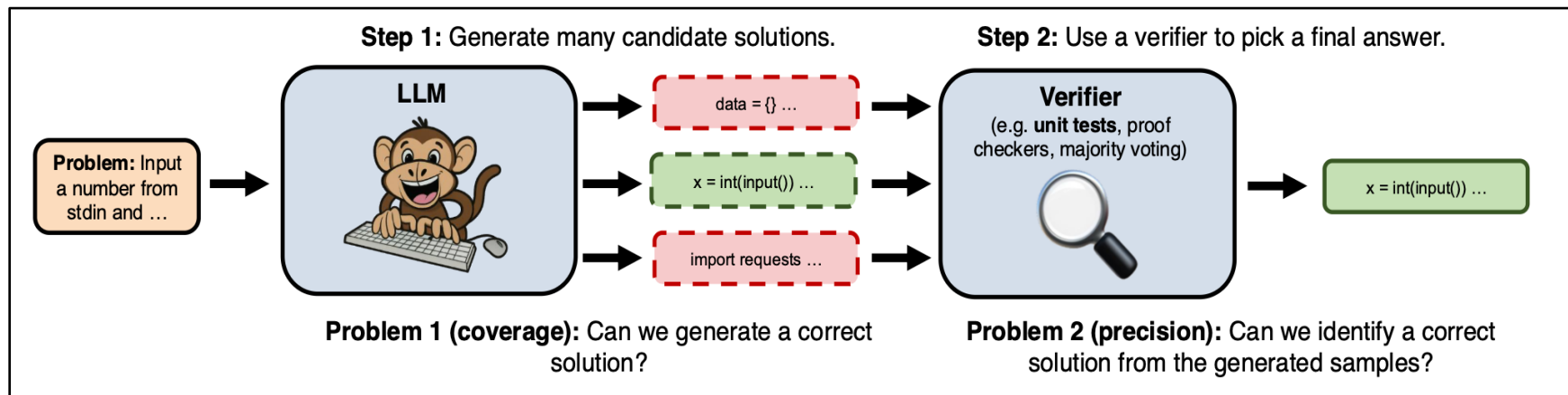
- A. Use the latest model.
- B. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc.
- C. Articulate the desired output format through examples.
- D. All of the above statements.

Which of the following suggestions is correct about prompt engineering? **C**

- A. A prompt must include the components of the instruction, the context, the input data, and the output indicator.
- B. A few-shot prompt is always better than a zero-shot prompt.
- C. In a prompt, it is recommended to tell the model what to do instead of what not to do.
- D. Chain-of-thought prompting can only be combined with few-shot prompts.

Inference Time Scaling

- With repeated sampling scenarios, we consider:
 - **Coverage**: As the number of samples increases, what fraction of problems can we solve using any sample that was generated?
 - **Precision**: In settings where we must select a final answer from the collection of generated samples, can we identify the correct samples?



- The log of coverage c as a function of the number of samples k :
 - $\log(c) = ak^{-b}$
 - $c = \exp(ak^{-b})$
- where $a, b \in \mathbb{R}$ are fitted model parameters.

Inference Time Scaling

- Inference scaling law indicates that we can scale the inference-time compute by sampling multiple branches in the solution space for performance boost.
- Consistency-based selection: a simple, effective, and general principle.
 - Revisit: self-consistency;
 - Universal self-consistency;
 - Train LLM ranker/verifier (ORM, PRM).
- When LLM self-evaluation works well: search in the partial solution space can help.

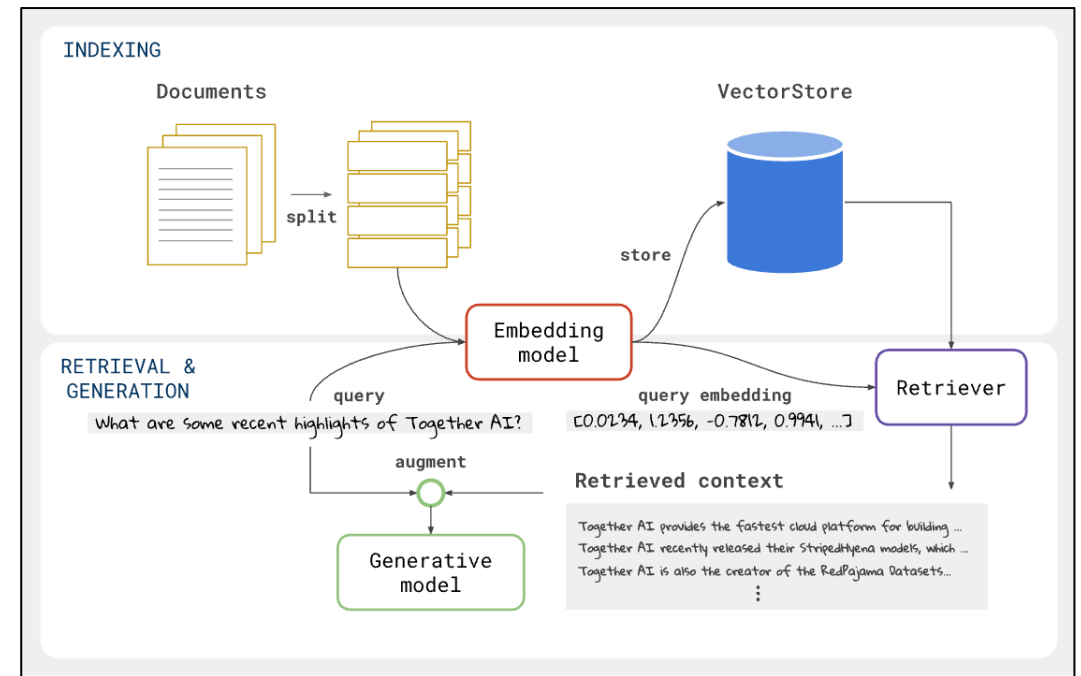
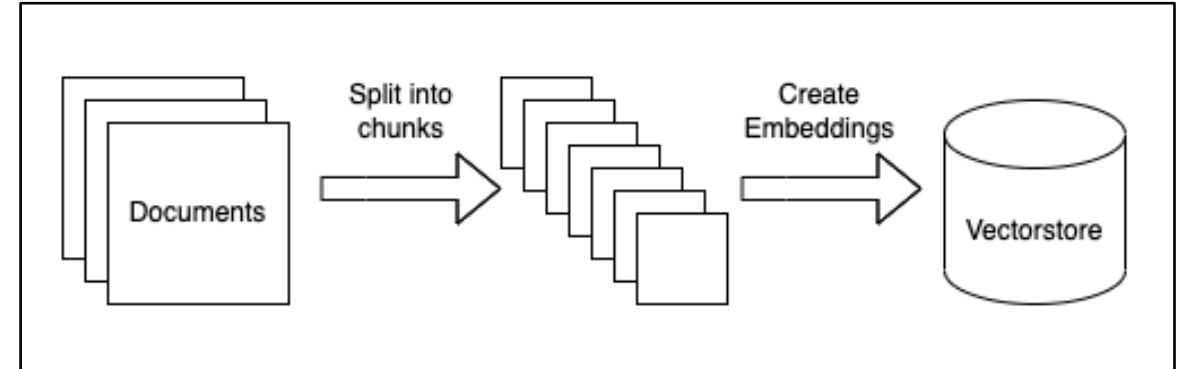
Retrieval-Augmented Generation

- **RAG Indexing:**

- Load data sources to text.
- Chunk text.
- Embed text.
- Load embeddings to vectorDB.

- **Retrieval and Generation:**

- Generate the embedding of the input prompt.
- Lookup relevant documents.
- Augment the prompt.
- Generate a response.



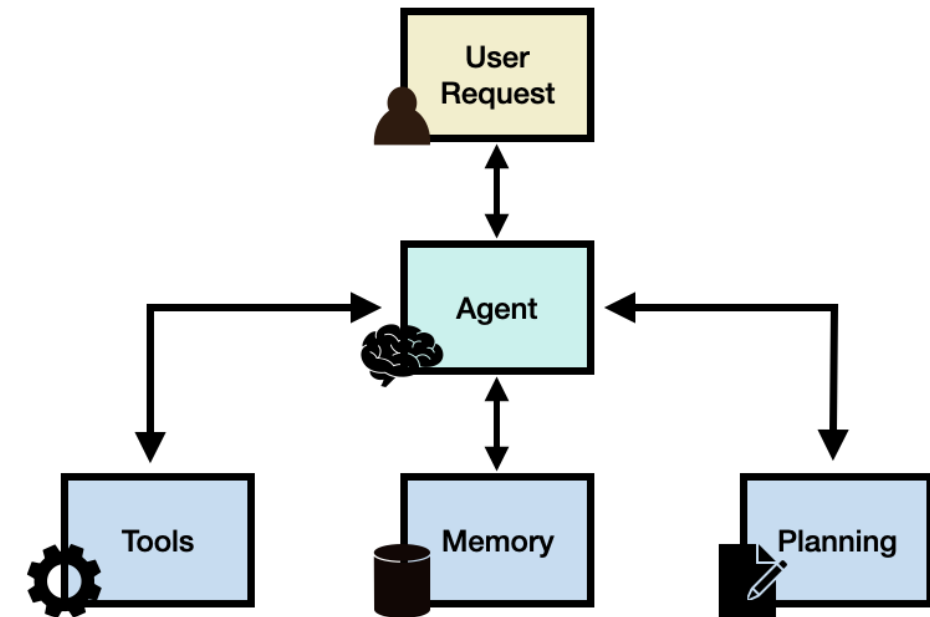
Sample Question

Which of the following suggestions is NOT correct about retrieval augmented generation? **B**

- A. The loaded data source must be split into smaller chunks to build the index.
- B. The chunked text is given to the embedding model to generate the embedding and stored in a vector database, where for each key-value pair, the key is the chunked text, and the value is the generated embedding.
- C. During the retrieval and generation phase, the embedding model is also needed.
- D. The retrieved data is integrated into the input prompt for the generated inference workflow.

LLM Agent Framework

- LLM applications that can execute complex tasks:
 - Through the use of LLMs;
 - And other key modules like planning and memory.
- The architecture of an LLM Agent framework can be very flexible, we just provide one simple yet clear categorization, which includes:
 - **User Request**: a user question or request;
 - **Agent/Brain**: the agent core acting as coordinator;
 - **Planning**: assists the agent in planning future actions;
 - **Memory**: manages the agent's past behaviours;
 - **Tool**: interacts with external environments.
- Examples: GUI Agent, Coding Agent.



LLM Alignment after Pre-Training

Parameter Efficient Fine-Tuning

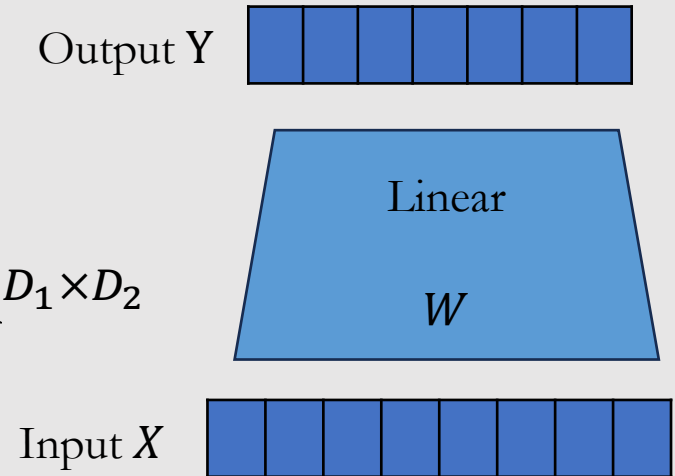
- Parameter efficient fine-tuning (PEFT): Rather than finetuning the entire model, we finetune only small amounts of weights.
 - Frozen layer/Subset fine-tuning: pick a subset of the parameters, fine-tune only those layers, and freeze the rest of the layers.
 - Adapters: add additional layers that have few parameters and tune only the parameters of those layers, keeping all others fixed.
 - Low-rank adaption (LoRA): learn a low-rank approximation of the weight matrices.

LoRA

- Keep the original pre-trained parameters W fixed during fine-tuning;
- Learn an additive modification to those parameters ΔW ;
- Define ΔW as a low-rank decomposition: $\Delta W = AB$.

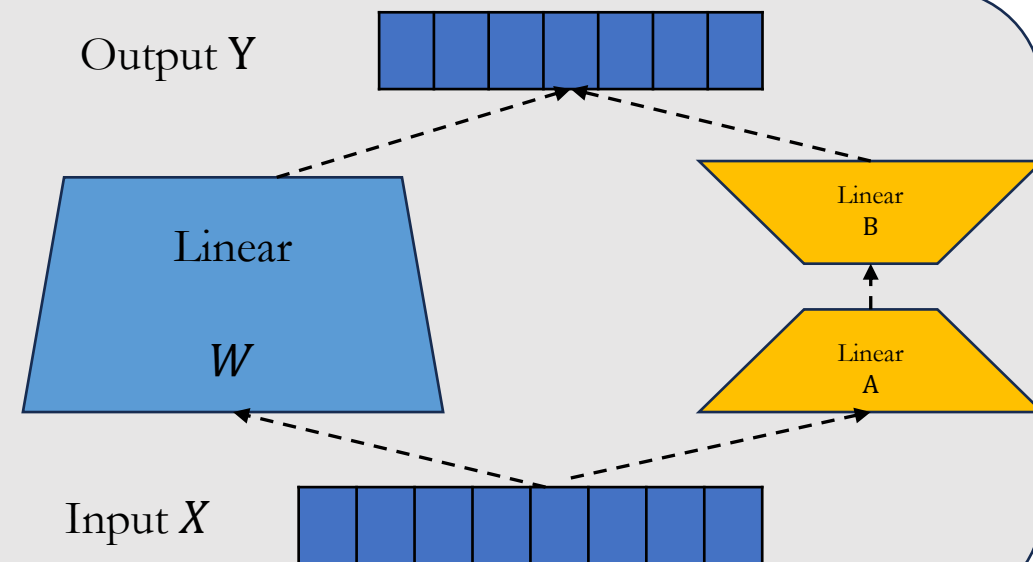
Standard Linear Layer

- $Y = XW$
- $X \in \mathbb{R}^{D_1}, Y \in \mathbb{R}^{D_2}, W \in \mathbb{R}^{D_1 \times D_2}$



LoRA Linear Layer

- $Y = XW + XAB = X(W + AB)$
- $X \in \mathbb{R}^{D_1}, Y \in \mathbb{R}^{D_2}, W \in \mathbb{R}^{D_1 \times D_2}$
- $A \in \mathbb{R}^{D_1 \times R}, B \in \mathbb{R}^{R \times D_2}$



Sample Question

Suppose we are adding LoRA layers to some fully connected (FC) layers in a neural network. The original computation of the FC layer can be summarized as:

- Forward computation:
 - $\mathbf{Y} = \mathbf{X}\mathbf{W}$, where given the input activation $\mathbf{X} \in \mathbb{R}^{B \times D}$ and the weight matrix $\mathbf{W} \in \mathbb{R}^{D \times D}$, we compute the output $\mathbf{Y} \in \mathbb{R}^{B \times D}$.
- Backward computation:
 - $\frac{dL}{d\mathbf{X}} = \frac{dL}{d\mathbf{Y}} \mathbf{W}^T$, where given the gradients w.r.t the output $\frac{dL}{d\mathbf{Y}} \in \mathbb{R}^{B \times D}$ and the weight matrix $\mathbf{W} \in \mathbb{R}^{D \times D}$, we compute the gradient w.r.t the input $\frac{dL}{d\mathbf{X}} \in \mathbb{R}^{B \times D}$.
 - $\frac{dL}{d\mathbf{W}} = \mathbf{X}^T \frac{dL}{d\mathbf{Y}}$, where given the gradients w.r.t the output $\frac{dL}{d\mathbf{Y}} \in \mathbb{R}^{B \times D}$ and the input $\mathbf{X} \in \mathbb{R}^{B \times D}$, we compute the gradients w.r.t the weight $\frac{dL}{d\mathbf{W}} \in \mathbb{R}^{D \times D}$.

The LoRA layer can be viewed as the stack of two FC layers:

- Forward computation:
 - $\mathbf{Z} = \mathbf{X}\mathbf{W}_A$, where given the input activation $\mathbf{X} \in \mathbb{R}^{B \times D}$ and the weight matrix $\mathbf{W}_A \in \mathbb{R}^{D \times R}$, we compute the intermediate output $\mathbf{Z} \in \mathbb{R}^{B \times R}$.
 - $\mathbf{Y}' = \mathbf{Z}\mathbf{W}_B$, where given the intermediate output $\mathbf{Z} \in \mathbb{R}^{B \times R}$ and the weight matrix $\mathbf{W}_B \in \mathbb{R}^{R \times D}$, we compute the layer output $\mathbf{Y}' \in \mathbb{R}^{B \times D}$.

Q1. Specify the backward propagation computation w.r.t the computation $\mathbf{Y}' = \mathbf{Z}\mathbf{W}_B$. (Be explicit about the dimension of the matrix following the above formalization of the FC layer).

- $\frac{dL}{d\mathbf{Z}} = \frac{dL}{d\mathbf{Y}'} \mathbf{W}_B^T$, where given the gradients w.r.t the output $\frac{dL}{d\mathbf{Y}'} \in \mathbb{R}^{B \times D}$ and the weight matrix $\mathbf{W}_B \in \mathbb{R}^{R \times D}$, we compute the gradient w.r.t the input $\frac{dL}{d\mathbf{Z}} \in \mathbb{R}^{B \times R}$.
- $\frac{dL}{d\mathbf{W}_B} = \mathbf{Z}^T \frac{dL}{d\mathbf{Y}'}$, where given the gradients w.r.t the output $\frac{dL}{d\mathbf{Y}'} \in \mathbb{R}^{B \times D}$ and the input $\mathbf{Z} \in \mathbb{R}^{B \times R}$, we compute the gradients w.r.t the weight $\frac{dL}{d\mathbf{W}_B} \in \mathbb{R}^{R \times D}$.

Q2. Specify the backward propagation computation w.r.t the computation $\mathbf{Z} = \mathbf{X}\mathbf{W}_A$. (Be explicit about the dimension of the matrix following the above formalization of the FC layer).

- $\frac{dL}{d\mathbf{X}} = \frac{dL}{d\mathbf{Z}} \mathbf{W}_A^T$, where given the gradients w.r.t the output $\frac{dL}{d\mathbf{Z}} \in \mathbb{R}^{B \times R}$ and the weight matrix $\mathbf{W}_A \in \mathbb{R}^{D \times R}$, we compute the gradient w.r.t the input $\frac{dL}{d\mathbf{X}} \in \mathbb{R}^{B \times D}$.
- $\frac{dL}{d\mathbf{W}_A} = \mathbf{X}^T \frac{dL}{d\mathbf{Z}}$, where given the gradients w.r.t the output $\frac{dL}{d\mathbf{Z}} \in \mathbb{R}^{B \times R}$ and the input $\mathbf{X} \in \mathbb{R}^{B \times D}$, we compute the gradients w.r.t the weight $\frac{dL}{d\mathbf{W}_A} \in \mathbb{R}^{D \times R}$.

Q3. Calculate the total number of float point operations in the forward and backward computation of the original linear layer defined in the problem.

- Forward: $2BD^2$.
- Backward: $4BD^2$.
- Total: $6BD^2$.

Q4. Calculate the total number of float point operations in the forward (defined by the problem) and backward computation (formulated by 2.1. and 2.2.) of the LoRA layer.

- Forward: $4BDR$.
- Backward: $8BDR$.
- Total: $12BDR$.

Q5. When combining the original FC layer with the LoRA layer, one needs to combine both outputs ($\mathbf{Y}$ from the original FC layer and $\mathbf{Y}'$ from the original LoRA layer), i.e., $\mathbf{Y} = \mathbf{Y} + \mathbf{Y}'$. Please calculate the total computation load in terms of float point operations for both LoRA and full parameter fine-tuning of the original FC layer. Assuming $R \ll D$, what is the ratio of computation load between LoRA and the full parameter efficiency (standard training of the FC layer)?

- LoRA total: $12BDR + 4BD^2$ (LoRA does not need to compute $\frac{dL}{d\mathbf{W}}$, but it needs to compute $\frac{dL}{d\mathbf{X}}$ in the original layer.)
- Full parameter finetune total: $6BD^2$.
- Ratio: $\frac{12BDR + 4BD^2}{6BD^2} = \frac{6R + 2D}{3D} \approx \frac{2}{3}$.

Homework 4-1

Memory Efficiency Estimation for Fine-Tuning Methods

1.1 Suppose we are using freezing layer fine-tuning. We only update the parameters in N_{top} transformer layers of the above model. How many bytes must be allocated to store the gradients w.r.t parameters in this computation? [15 points]

Answer: $24 \times N_{\text{top}} \times D^2$

1.2 Suppose we are using adapter fine-tuning. Suppose the intermediate dimension in each adapter layer is R . We are adding two adapter layers in each transformer layer defined above. How many bytes must be allocated to store i) the additional parameters in adapter layers and ii) the gradients w.r.t these additional parameters in this computation? [15 points]

Answer:

i): $2 \times 2 \times N_{\text{layer}} \times (D \times R + R \times D) = 8N_{\text{layer}}DR$

ii): the same as i).

1.3 Suppose we are using LoRA fine-tuning. Suppose the intermediate dimension in each LoRA adapter layer is R . We only apply LoRA to query, key, and value layers in each transformer layer defined above. How many bytes must be allocated to store i) the additional parameters in LoRA layers and ii) the gradients w.r.t these additional parameters in this computation? [15 points]

Answer:

i): $2 \times 3 \times N_{\text{layer}} \times (D \times R + R \times D) = 12N_{\text{layer}}DR$

ii): the same as i).

1.4 Suppose we are using VeRA fine-tuning. Suppose the intermediate dimension in each VeRA adapter layer is R . We only apply LoRA to query, key, and value layers in each transformer layer defined above. How many bytes must be allocated to store i) the additional tunable parameters in VeRA layers and ii) the gradients w.r.t these additional tunable parameters in this computation? [10 points]

i): $2 \times 3 \times N_{\text{layer}} \times (D + R) = 6N_{\text{layer}}(D + R)$

ii): the same as i).

RL Alignment for LLM

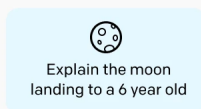
- Under the context of LLM alignment by RL:
 - **State**: The state corresponds to the input prompt provided to the language model. In RL terms, this is the observation that informs the agent's next action.
 - **Action**: The action is the response generated by the LLM in reaction to the input prompt. Each token produced can be considered an individual action, making the entire response a sequence of actions. This sequential generation aligns with the token-by-token decision-making process in reinforcement learning.
 - **Policy**: The policy in this context is the LLM itself, which maps input prompts (states) to probability distributions over possible next tokens (actions).
 - **Reward**: The reward is a scalar value representing the quality of the generated response, which guides the policy updates, encouraging the model to produce outputs that align with human expectations.

RLHF

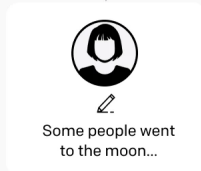
Step 1

Collect demonstration data, and train a supervised policy.

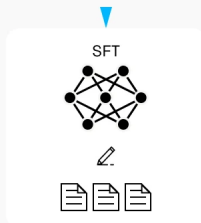
A prompt is sample from our prompt dataset.



A labeler demonstrates the desired output behavior.



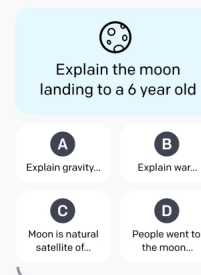
This data is used to fine-tune GPT-3 with supervised learning.



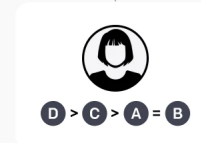
Step 2

Collect comparison data, and train a reward model.

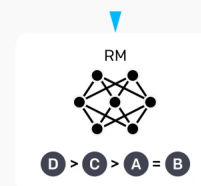
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



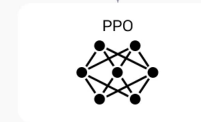
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

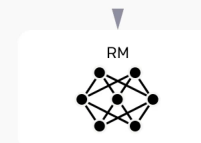


The policy generates an output.

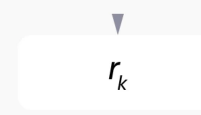


Once upon a time...

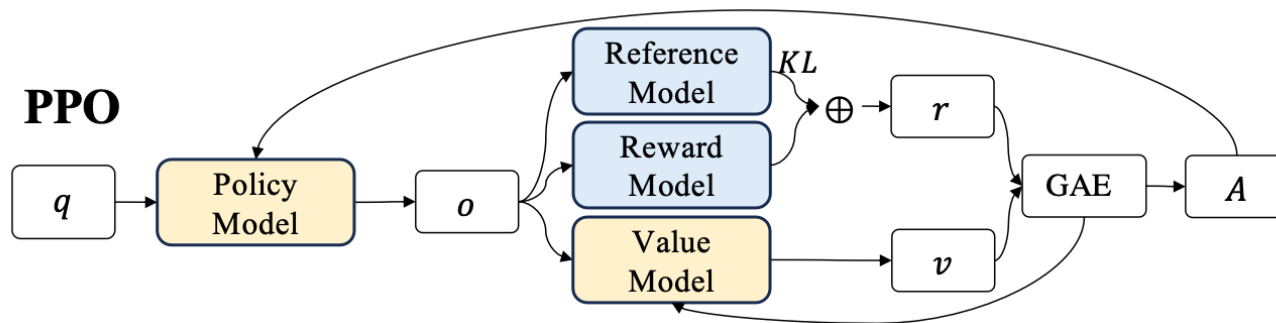
The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



PPO



LLM Evaluation

Quality Evaluation Format and Metric

- Multiple-Choice Question (MCQ):
 - Evaluation metric: *accuracy*, i.e. , the percentage of questions answered correctly.
- Coding Problem:
 - Evaluation metric: *pass@k* e.g., if at least one of k tries passes all tests.
- Open-ended Question:
 - Evaluation method 1- exact match: *accuracy*.
 - Evaluation method 2 – partial match: *precision, recall, F1 score*.
 - Evaluation method 3 - human evaluation: *pairwise comparisons (Leaderboard ChatArena)*
 - Evaluation method 4 – LLM-as-a-Judge: *pairwise comparison, single output scoring (w/wo reference-answer)*.

Representative Benchmarks

- General-purpose benchmark:
 - MMLU (MMLU-Pro, MMMLU), GPQA, SimpleQA, SimpleQA-Verified.
- Coding benchmark:
 - CodeContests, SWE-Bench.
- Math benchmark:
 - Math-500, GSM8K, AIME.

Final Exam Arrangement

- Date: May 29, 2026 (Friday)
- Time:
 - 120 minutes, 16:30-18:30 (You can leave earlier if you want.)
- Location:
 - LG5 Multi-function Room
- Bring:
 - Your student ID;
 - One page of your cheating sheet in A4 size, printed or hand-written.
- Do NOT use a pencil in the exam. Otherwise, you are not allowed to appeal for any grading disagreements!
- The HKUST Academic Honor Code applies! I am very serious about this!
- Absence:
 - I must be informed, and your appeal must be confirmed by email before the exam starts.

Good Luck for the Final!

Please Submit the Course Feedback



Thank you!

<https://sfq-survey.ust.hk/>